

Bowling Game Features and Implementation Logic

1. GameManager (Bowling Game Manager)

1.1 Game Area Setup

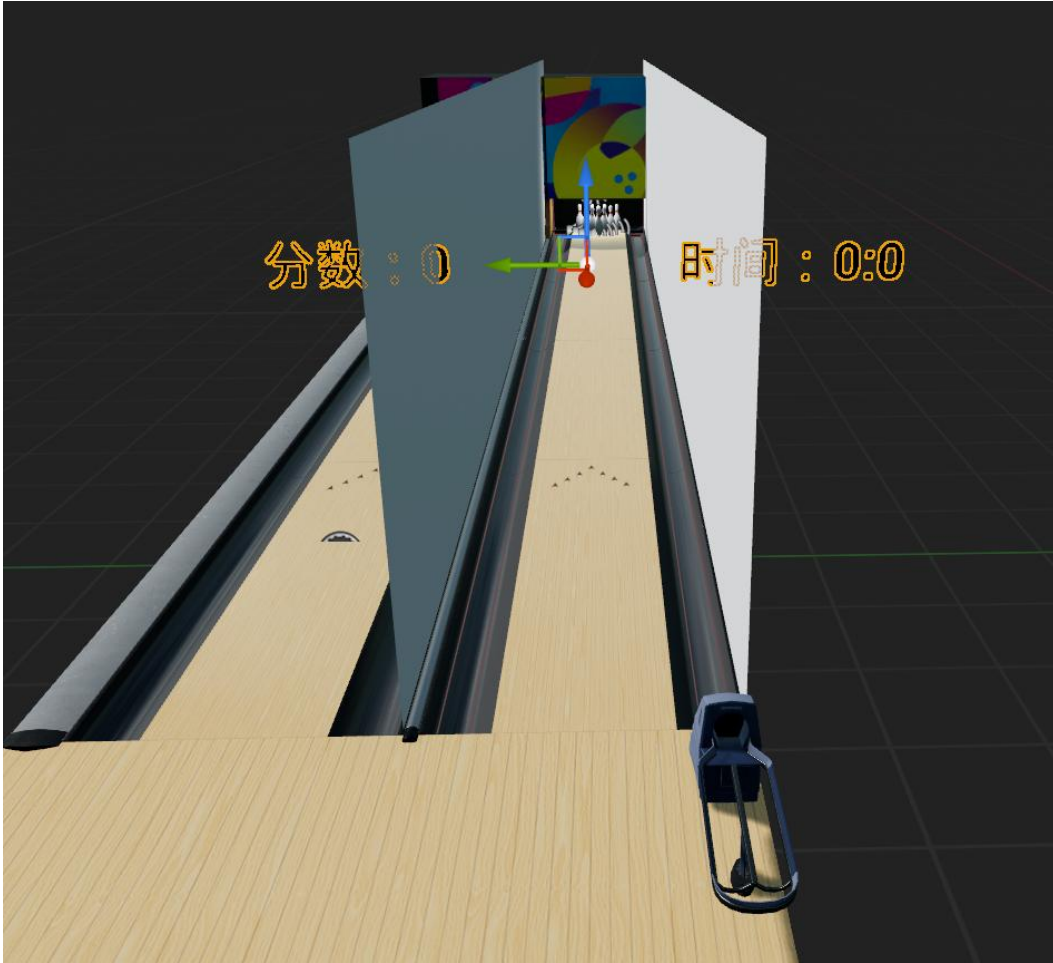
Purpose

Set up the bowling game area, including the lane, bowling pins, ball return machine, score and time display, and invisible walls on both sides to keep bowling balls from flying out.

Description

- Lane: the floor uses a physical material so the bowling ball can roll on it
- Bowling pins: 10 pins manually placed in the standard triangle formation at the end of the lane
- Ball return machine: located on the player side in front of the lane; the bowling ball spawns on it and the player picks the ball up from it
- Invisible walls: Plane colliders on both sides block bowling balls from flying out; with Hide in Game checked they are invisible in game but keep their collision

Blueprint screenshots

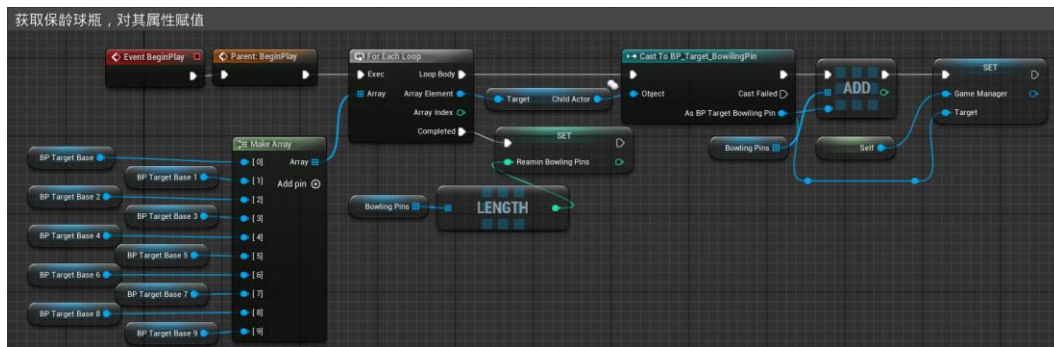


1.2 Bowling Pin Initialization

Purpose

At game start, gets references to the 10 manually placed bowling pins in the level, stores them in an array for later use, and sets the GameManager reference on each pin.

Blueprint screenshots



Key points

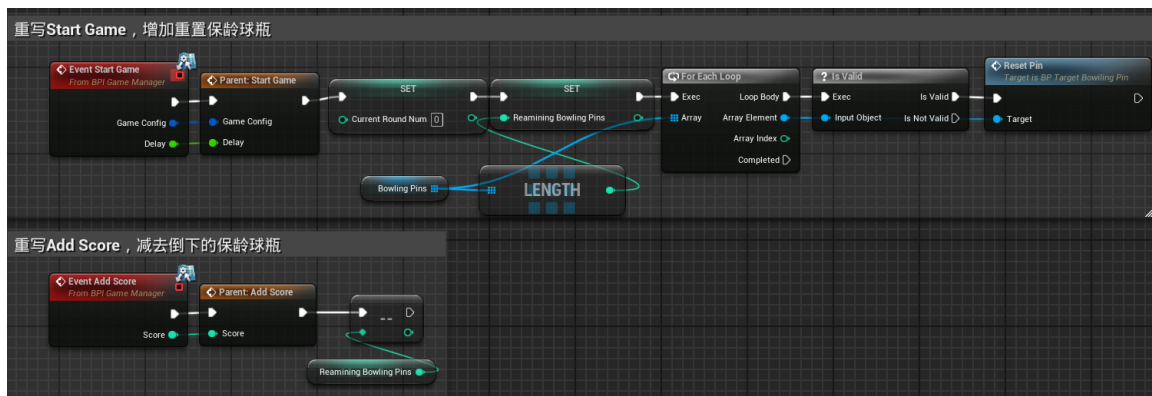
- A Cast ensures only pins of type BP_Target_BowlingPin are added to the array
- Each pin gets the GameManager reference so it can call the GameManager's AddScore function
- Remaining Bowling Pins is initialized to the total number of pins (10) for later round decisions

1.3 StartGame and AddScore Overrides

Purpose

Overrides the parent class's StartGame and AddScore functions: StartGame resets all bowling pins to their initial state and the incremented data; AddScore updates the remaining pin count.

Blueprint screenshots



Key points

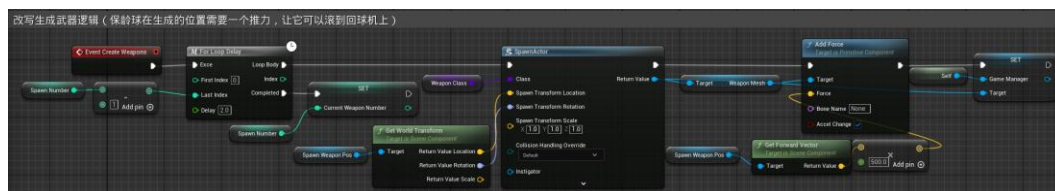
- StartGame calls Reset Pin to ensure all pins are standing when the game starts

1.4 Weapon Spawn Logic (Spawning the Bowling Ball)

Purpose

Overrides the parent class's CreateWeapons logic: spawns the bowling ball at the ball return machine and applies a forward force so it rolls onto the machine for the player to pick up. Only one ball is spawned to avoid confusion from multiple balls at once.

Blueprint screenshots



Key points

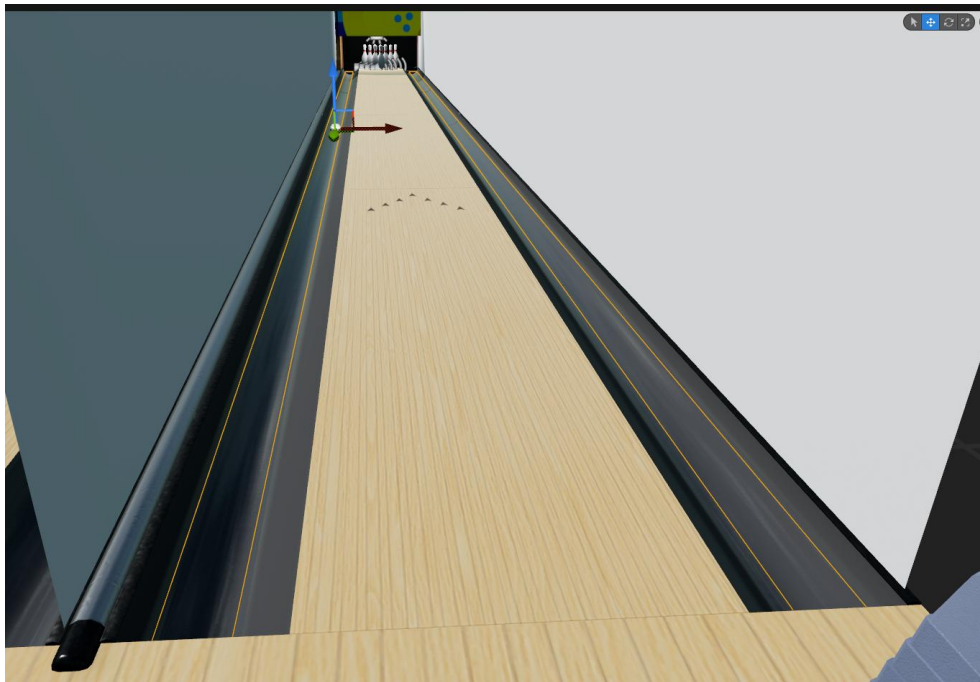
- Spawn Number is set to 1 so only one bowling ball is on the lane at a time
- Add Force uses Get Forward Vector $\times 500$, giving the ball a forward push so it rolls onto the ball return machine
- The spawned bowling ball automatically receives the GameManager reference for later interaction

1.5 Side Gutter Acceleration Zones

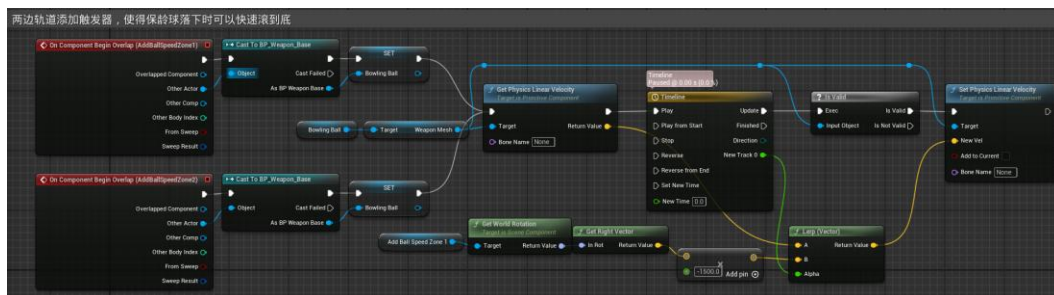
Purpose

Triggers (AddBallSpeedZone1 and AddBallSpeedZone2) are placed on the side gutters; when the bowling ball rolls into a gutter, an acceleration is applied so it quickly rolls to the bottom, preventing a slow-rolling ball in the gutter from hurting the experience.

Scene screenshots



Blueprint screenshots



Key points

- Two symmetric triggers handle the left and right gutters separately
- Add to Current is checked to add to the existing velocity, preserving the ball's momentum
- $-1500.0 \times \text{Right Vector}$ accelerates the ball toward the front of the gutter (the bottom)

1.6 Bottom Settlement Logic

Purpose

When the bowling ball rolls to the bottom of the lane and enters the destroy zone (DestroyZone), the throw is settled: hide the bowling ball, hide the knocked-down pins, decide whether to reset the pins, consume an attempt, and check whether the game ends.

Execution flow

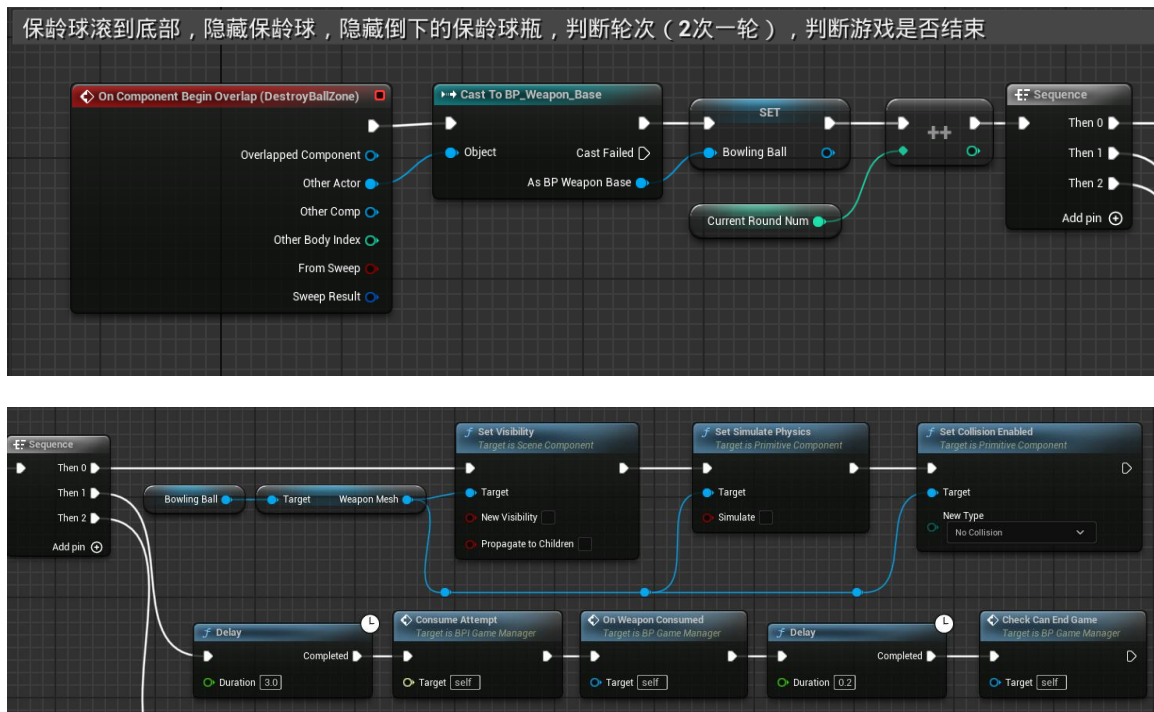
- On Component Begin Overlap (DestroyZone) fires, detecting the bowling ball entering the bottom area
- Sequence runs multiple paths in parallel:

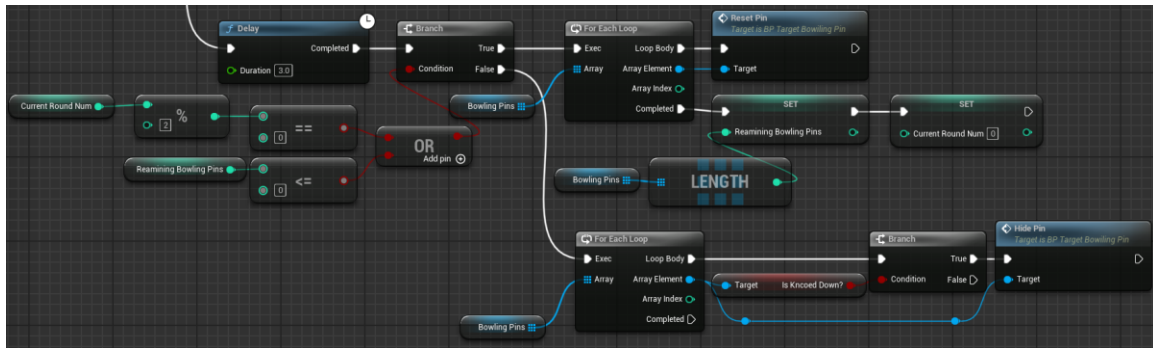
Path 1: hide the bowling ball (executed immediately)

Path 2: consume the attempt and check the end (after a 3.0-second Delay)

Path 3: settle the bowling pins (after a 3.0-second Delay)

Blueprint screenshots





Key points

- The 3.0-second Delay ensures all pins finish their physics settlement before judging their state, avoiding misjudgment
- Reset condition: after 2 throws (one round) or when all pins are down, all pins are reset
- Non-reset condition: only the knocked-down pins are hidden; standing pins stay for the next ball
- The bowling ball's attempt consumption and end check are called in the GameManager, not at throw time, because a throw only completes when the ball reaches the bottom

Notes

- The reset and hide logic are mutually exclusive: either reset all pins or only hide the knocked-down ones
- Current Round Num decides whether the current ball is the first or second, controlling whether a reset is needed

2. BP_Target_BowlingPin (Bowling Pin)

2.1 Overview

Purpose

BP_Target_BowlingPin inherits from BP_Target_Base and is the bowling game's scoring target. Two booleans (IsKnocked and IsKnockedDown) separately track the "was touched" and "has fallen" states, and Event Tick continuously checks the tilt angle so a pin is scored and hidden correctly when it falls. The Reset Pin function restores fallen pins to their initial position and state.

Description

- On Hit event: detects the collision and sets IsKnocked = true (the dual booleans prevent duplicates)

- Event Tick: continuously checks the tilt angle to determine whether the pin has truly fallen
- Reset Pin function: resets the pin to its initial position, rotation, and state
- Hide Pin event: hides the pin mesh, used to hide fallen pins during settlement
- BeginPlay: records the pin's initial Transform (Pin Start Transform) for the reset

2.2 Dual-Boolean Design

Core principles

Two independent booleans represent the pin's different state stages, preventing double scoring and logic confusion:

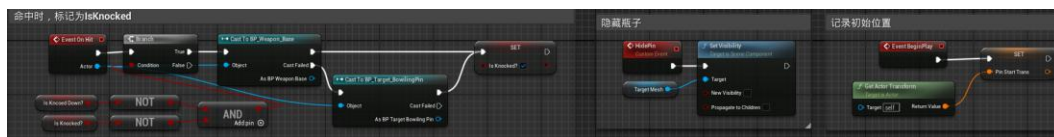
Variable	Default	Meaning	When set
IsKnocked	false	Whether the pin was touched	Set to true in On Hit (requires IsKnockedDown to be false); set to false in Tick after falling
IsKnockedDown	false	Whether the pin has fallen	Set to true in Tick when the tilt exceeds the threshold; set to false in Reset Pin

Design

- In On Hit: IsKnocked is set to true only when both IsKnocked and IsKnockedDown are false. A fallen pin (IsKnockedDown = true) is never marked as touched again
- In Tick: once the pin is detected as fallen, IsKnockedDown is set to true and IsKnocked to false so Tick no longer repeats the fall logic, preventing double scoring
- The dual-boolean design guarantees each pin is scored only once and is not repeatedly scored by the continuously running Tick

2.3 On Hit Event (Collision Marking)

Blueprint screenshots

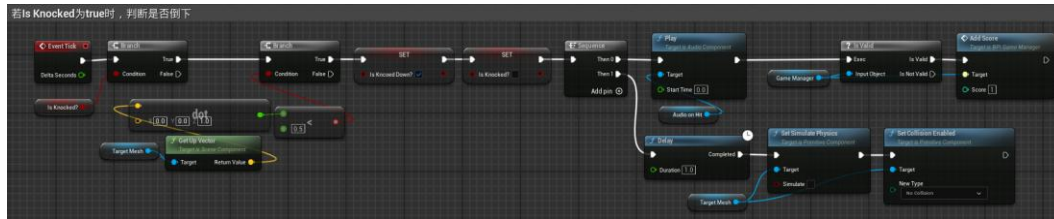


Key points

- A Cast filters so only collisions involving the bowling ball (weapon) or bowling pins trigger the marking
- The NOT IsKnockedDown check ensures fallen pins are not marked again
- On Hit only marks the "was touched" state; it does not run scoring or hiding logic

2.4 Tick Fall Detection (Core Logic)

Blueprint screenshots



Core principles

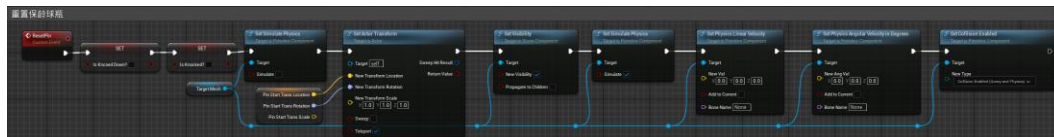
- Tilt detection principle: the dot product of the Up Vector with the world Z axis = $\cos(\text{tilt angle})$. A dot product of 1.0 is fully upright, 0.5 is tilted 60 degrees, and below 0 is lying flat
- Threshold 0.5: a pin tilted more than 60 degrees is judged as fallen, matching how real bowling pins actually fall
- 1.0-second Delay: after falling the pin stays visible for 1 second so the player sees it fall, then it is hidden and physics disabled

2.5 Reset Pin Function (Resetting the Pin)

Purpose

Resets a fallen or hidden bowling pin to its initial position, rotation, and state for the next round of throws.

Blueprint screenshots



Key points

- Teleport = true is the key setting: it instantly teleports the Actor to the target position without collision detection or physics response. Without it, the engine checks collision during the move, and a fallen pin righting itself could be blocked by the ground or other pins, failing to reach the correct position
- Disable physics before setting the position: prevents the physics engine from producing unexpected collision responses during the position change
- Re-enable physics after setting the position: restores the pin's physics simulation so it can respond to the bowling ball's collisions
- Zero the velocity: prevents the pin from keeping its previous momentum and moving unexpectedly after the reset
- Restore collision: ensures the reset pin can be collided with and detected by the bowling ball

2.6 Physics Property Settings

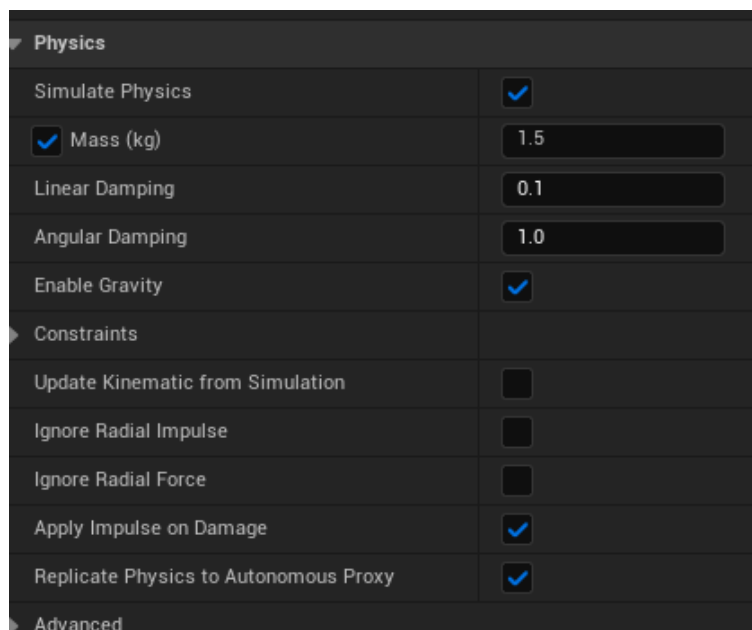
Purpose

Configures the bowling pin's physical properties so its collision and falling behavior is natural; the high angular damping in particular prevents pins from slowly rotating over after a light collision.

Physics property configuration

Property	Value	Description
Simulate Physics	Enabled	The pin is driven by physics simulation and can be knocked down by collisions
Mass	1.5 kg	Close to a real bowling pin's mass (1.5–1.6 kg)
Linear Damping	0.1	Low linear damping so knocked-down pins slide naturally
Angular Damping	1.0	High angular damping preventing slow toppling from light collisions and reducing excessive spinning after being hit
Enable Gravity	Enabled	The pin is affected by gravity and lands naturally when falling

Blueprint screenshots



Design

- Angular Damping = 1.0: the small angular velocity from a light collision decays quickly, so the pin either falls within a very short time (detected by Tick) or returns to a stable upright state — it never slowly topples seconds later
- The high angular damping also keeps directly hit pins from spinning forever, letting them stop within a reasonable time

3. BP_Weapon_BowlingBall (Bowling Ball)

3.1 Overview

Purpose

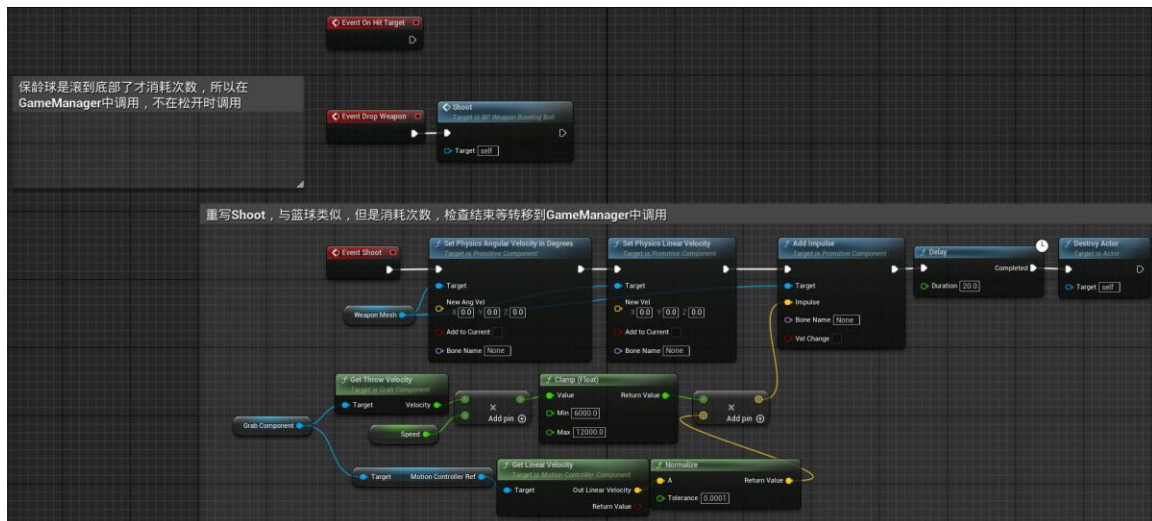
BP_Weapon_BowlingBall inherits from BP_Weapon_Base and implements a physics-impulse-based VR throwing mechanism. It is similar to the basketball, but attempt consumption and the game-end check are moved into the GameManager because a bowling throw only completes when the ball reaches the bottom.

Description

- Shoot event override: applies an impulse with Add Impulse, consistent with the basketball throw logic
- OnHitTarget event override: the bowling ball does not need to destroy itself on hitting a target, so it is overridden as a no-op
- Attempt consumption and end check: not called at throw time; moved into the GameManager's bottom settlement logic

3.2 Shoot Event Execution Flow

Blueprint screenshots



Core principles

- Zeroing velocity: when the bowling ball is released from a grab, the physics system leaves residual linear and angular velocity. Zero them before applying the impulse so the ball moves only along the throw direction, unaffected by residual momentum
- Vel Change unchecked: the impulse is mass-affected ($\text{impulse} / \text{mass} = \text{velocity change}$). The bowling ball's mass is 7 kg and the impulse is 6000–12000, giving an actual velocity of 857–1714 cm/s (8.5–17 m/s), close to a real bowling ball's speed
- Clamp limit: the 6000–12000 range keeps throw strength within a sensible window, preventing too-light throws that can't reach the bottom or too-hard throws that cause physics anomalies

4. Complete Bowling Flow

Execution flow

The complete flow from bowling ball spawn to settlement:

1. The GameManager calls CreateWeapons, spawns one bowling ball at the ball return machine, and applies a push so it rolls onto the machine
2. The player's VR controller approaches the bowling ball, the Grab Component triggers a grab, and the ball attaches to the controller
3. The player makes a throwing motion and releases the trigger, firing the Shoot event
4. The bowling ball's angular and linear velocities are zeroed, the throw direction and impulse are computed, and Add Impulse is applied
5. The bowling ball rolls down the lane; if it enters a side gutter, a trigger applies an acceleration so it quickly rolls to the bottom
6. The bowling ball strikes the pins, each pin's On Hit event fires, and IsKnocked = true is marked
7. A pin may be knocked down directly or toppled by other pins (chain reaction)
8. Every touched pin continuously checks its tilt in Tick and is judged fallen when the tilt exceeds 60 degrees
9. A fallen pin: play sound → AddScore(1) → Delay 1.0 s → disable physics → disable collision
10. The bowling ball keeps rolling to the bottom of the lane and enters the DestroyZone trigger
11. The GameManager hides the bowling ball and waits 3.0 seconds for the physics to settle
12. Decide whether to reset: if the round is over or all pins are down → Reset Pin resets all pins; otherwise → Hide Pin hides the fallen pins
13. Consume Attempt consumes the throw count and Check Can End Game checks whether the game ends
14. If the game is not over, the GameManager spawns a new bowling ball and the next throw begins